

Problem D. BitMazeCraft

Time Limit 3000 ms

Mem Limit 262144 kB

OS Windows

Vlad is a very responsible employee. But instead of working he plays *BitMazeCraft*. But not the survival mode, he prefers completing user-created maps. As you might guess, it's very difficult. Especially when Vlad's co-workers interrupt him constantly.

Today Vlad is trying to complete a map of size $A \times B \times H$, where H is the map's height. He needs to move the character from one spot to another without leaving the map's boundaries. The map consists of $1 \times 1 \times 1$ cubes, each cube has coordinates (x, y, z) and is either occupied by a wall or empty ($1 \leq x \leq A, 1 \leq y \leq B, 1 \leq z \leq H$). Vlad is a bit anxious about his boss showing up, so he wants to complete the map as soon as possible. To "complete the map" means to move the character from start to finish.

Let's say that a cell is located *above the floor* if either its z coordinate is equal to 1 or the cube below it is occupied. The controls in *BitMazeCraft* are simple: the character has size $1 \times 1 \times 1$ and it must be located in an empty cell above the floor at all times. The character has four types of movement, every one takes up the same amount of time.

- moving horizontally into a neighboring cell $((x, y, z) \rightarrow \{(x, y \pm 1, z), (x \pm 1, y, z)\})$ if the target cell is empty and above the floor (such movement is called *walk*)
- moving horizontally over the neighboring cell $((x, y, z) \rightarrow \{(x, y \pm 2, z), (x \pm 2, y, z)\})$ if both cells in the direction of movement are empty; the cell character passes over $((x, y \pm 1, z-1), (x \pm 1, y, z-1))$ is empty (otherwise the character would tumble); and the target cell is above the floor (such movement is called *jump*)
- descent by one layer while moving into a neighboring cell laterally $((x, y, z) \rightarrow \{(x, y \pm 1, z-1), (x \pm 1, y, z-1)\})$ if the target cell is empty and above the floor; and the cell above the target cell is also empty (such movement is called *drop*)
- ascent by one layer while moving into a neighboring cell laterally $((x, y, z) \rightarrow \{(x, y \pm 1, z+1), (x \pm 1, y, z+1)\})$ if the cell above the starting cell is within the boundaries of the map and empty; and the target cell is empty and above the floor (such movement is called *climb*).

In fact, Vlad isn't even sure if the map can be completed at all. So he asked you to check if the map can be completed, and if that's true, what's the fastest way to get from start to finish.

Input

The first line contains three space-separated integer numbers A, B, H — length, width and height of the map ($1 \leq A, B, H \leq 50$). Then, $H \cdot A$ lines of length B grouped into H blocks with A lines each are given.

Each block describes the slice of the map with a fixed z coordinate. They are given **from bottom to top**, in the ascending order of z . Each line describes a slice of a block with a fixed x coordinate, lines within a block are given in ascending order of x . Characters in a line describe cells and are given in ascending order of y . Character “#” (code 35) denotes an occupied cell, character “.” (code 46) denotes an empty cell.

The last two lines contain three space-separated integers each — starting point of the characted $x_{from}, y_{from}, z_{from}$ and the destination point x_{to}, y_{to}, z_{to} respectively ($1 \leq x_{from}, x_{to} \leq A, 1 \leq y_{from}, y_{to} \leq B, 1 \leq z_{from}, z_{to} \leq H$).

It is guaranteed that both the starting cell and the destination cell are empty and are above the floor.

Output

If it's impossible to complete the map, output “**NO**” (without quotes) in a single line.

Otherwise, output “**YES**” (without quotes) in the first line. In the second line, output an integer N — the minimal amount of moves to complete the level. In each of next N lines,

output the moves in the following format: $type_i direction_i$, where $type_i$ is the type of movement “**walk**”, “**jump**”, “**drop**” or “**climb**” (without quotes); $direction_i$ — the direction of movement “**north**”, “**east**”, “**south**” or “**west**” (without quotes):

north — moving from (x, y, z) to $(x - k, y, z')$, **east** — from (x, y, z) to $(x, y + k, z')$,

south — from (x, y, z) to $(x + k, y, z')$ or **west** — from (x, y, z) to $(x, y - k, z')$; where k is equal to 2 if $type_i$ is **jump** and 1 otherwise; z' equals to z if $type_i$ is **walk** or **jump**, $z - 1$ if $type_i$ is **drop** and $z + 1$ if $type_i$ is **climb**.

If there are many optimal ways to complete the map, you may output any one of them.

Sample

Input	Output
<pre>1 2 1 . . 1 1 1 1 2 1</pre>	<pre>YES 1 walk east</pre>

Input	Output
3 4 2 .#..#.. 1 1 1 3 4 1	YES 4 climb east jump south drop east walk east